

データ構造 区間クエリ Sqrt Tree Disjoint Sparse Table モノイド

# Sqrt Tree

## 1. 概要

本講座では、**Sqrt Tree** について解説します。

Sqrt Tree は列の平方分割を再帰的に行うことでできる木構造です。特に静的なモノイド列に対する区間積クエリを高速に処理できます。具体的には、列の長さを  $N$  とするとき、事前計算  $O(N \log \log N)$  時間のもと、クエリあたり  $O(1)$  時間で区間積を処理することができます。

区間積クエリの解法としてはセグメント木や Disjoint Sparse Table（以下 DST とも略記します）を用いる方法も有名ですが、クエリの個数  $Q$  が  $N$  に近い状況であれば、Sqrt Tree はセグメント木や DST よりも良い計算量オーダーを達成します。

## 2. 前提知識

セグメント木や Disjoint Sparse Table、モノイドについてある程度理解していると、本講座の内容を理解しやすくなります。必要であればモノイドについては 代数構造用語集 を参照してください。

## 3. 問題設定

本講座で扱う問題は、Disjoint Sparse Table **【問題 1】** と同一です。再掲します。

### 【問題 1】

$(M, *)$  をモノイドとします。  $M$  の元からなる長さ  $N$  の列  $A = (A_0, A_1, \dots, A_{N-1})$  が与えられます。  $Q$  個のクエリに答えてください。クエリでは  $0 \leq L < R \leq N$  を満

たす整数  $L, R$  が与えられるので,

$$A_L * A_{L+1} * \cdots * A_{R-1}$$

を出力してください.

また, 計算量解析に際しては,  $M$  の元に対する操作 (配列への読み書きやモノイド演算) が  $O(1)$  時間であるとします.

## 4. Sqrt Tree

### 4.1. 平方分割の利用

Sqrt Tree は, 列の平方分割を再帰的に行うことで得られる木構造です. まずは「再帰的に行う」ということを無視して, **単に平方分割を利用した場合**にできることを整理します.

ブロックサイズ  $S$  を  $S = \lfloor \sqrt{N} \rfloor$  程度にとり, 列全体を,  $K = \lceil N/S \rceil$  個のブロックに分割します.

ブロックに  $0, 1, \dots, K-1$  の番号をつけて, 以下のような 3 つのテーブルを事前計算します.

- Prefix : ブロック  $j$  に属する各  $A_i$  について, ブロック  $j$  の先頭から  $A_i$  までの総積を計算する.
- Suffix : ブロック  $j$  に属する各  $A_i$  について,  $A_i$  からブロック  $j$  の末尾までの総積を計算する.
- Between : 各ブロック  $(i, j)$  (ただし  $i \leq j$ ) について, ブロック  $i$  の先頭からブロック  $j$  の末尾までの総積を計算する.

$N = 12, S = 3$  の場合に, これらのテーブルにどのような区間の総積が格納されるかを図示すると次のようになります.

|         |          |          |          |           |          |          |          |          |          |           |            |            |
|---------|----------|----------|----------|-----------|----------|----------|----------|----------|----------|-----------|------------|------------|
| $A$     | $A_0$    | $A_1$    | $A_2$    | $A_3$     | $A_4$    | $A_5$    | $A_6$    | $A_7$    | $A_8$    | $A_9$     | $A_{10}$   | $A_{11}$   |
| Prefix  | $[0, 1)$ | $[0, 2)$ | $[0, 3)$ | $[3, 4)$  | $[3, 5)$ | $[3, 6)$ | $[6, 7)$ | $[6, 8)$ | $[6, 9)$ | $[9, 10)$ | $[9, 11)$  | $[9, 12)$  |
| Suffix  | $[0, 3)$ | $[1, 3)$ | $[2, 3)$ | $[3, 6)$  | $[4, 6)$ | $[5, 6)$ | $[6, 9)$ | $[7, 9)$ | $[8, 9)$ | $[9, 12)$ | $[10, 12)$ | $[11, 12)$ |
| Between | $[0, 3)$ | $[0, 6)$ | $[0, 9)$ | $[0, 12)$ |          |          |          |          |          |           |            |            |
|         |          | $[3, 6)$ | $[3, 9)$ | $[3, 12)$ |          |          |          |          |          |           |            |            |
|         |          |          | $[6, 9)$ | $[6, 12)$ |          |          |          |          |          |           |            |            |
|         |          |          |          | $[9, 12)$ |          |          |          |          |          |           |            |            |

これらの事前計算にかかる計算量は、

- Prefix :  $i$  について昇順に計算すれば、 $O(N)$  時間です。
- Suffix :  $i$  について降順に計算すれば、 $O(N)$  時間です。
- Between : Suffix により特に、各ブロック内の総積が求まっていることに注意します。  $i$  を固定するごとに  $j$  について昇順に計算すれば、 $O(N)$  時間（ペア  $(i, j)$  ごとに  $O(1)$  時間）です。

したがって、このような事前計算は  $O(N)$  時間で行えます。

以上の事前計算のもと、クエリに答えることを考えます。するとクエリ区間を  $[L, R)$  とするとき、 $A_L$  と  $A_{R-1}$  の属するブロックが異なるならばクエリに  $O(1)$  時間で答えられることが分かります。具体的には  $[L, R)$  を次のような 3 つの部分区間に分割します。

- $A_L$  から、 $A_L$  の属するブロックの末尾まで。
- $A_L$  の属するブロックと、 $A_{R-1}$  の属するブロックの間に挟まれたブロック全体。
- $A_{R-1}$  の属するブロックの先頭から  $A_{R-1}$  まで。

ただし、2 番目の「間に挟まれたブロック全体」は空なこともあります。

|       |       |       |       |       |       |       |       |       |       |          |          |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|
| $A_0$ | $A_1$ | $A_2$ | $A_3$ | $A_4$ | $A_5$ | $A_6$ | $A_7$ | $A_8$ | $A_9$ | $A_{10}$ | $A_{11}$ |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|

$$A_1 A_2 \cdots A_9 = \text{Suffix}[1] \times \text{Between}[1][3] \times \text{Prefix}[9]$$

これらの 3 つの区間の総積は、(空積でないならば) 事前計算した値に含まれています。よって、それらの値をテーブルから読み取ったあとモノイド演算 2 回を行うことで、区間クエリに答えられます。

ただし、クエリ区間  $[L, R)$  について、「 $A_L$  と  $A_{R-1}$  の属するブロックが異なる」という条件が必要だったことに注意してください。

## 4.2. Sqrt Tree

4.1 節の平方分割によって、

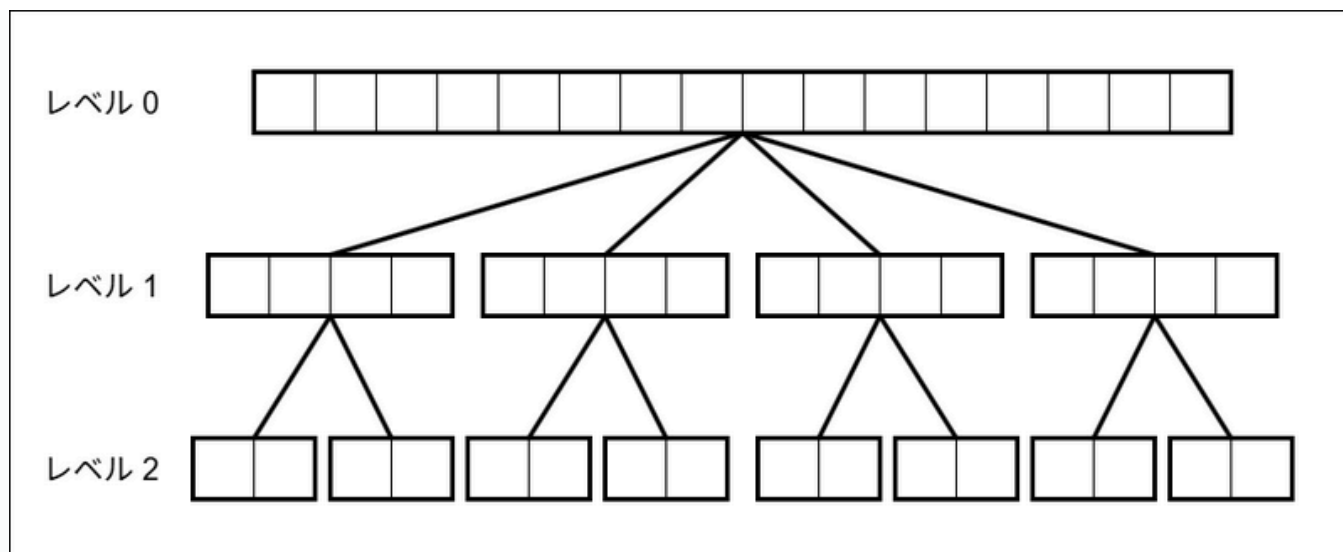
- 空間  $O(N)$ , 時間  $O(N)$  事前計算により
- クエリで与えられる区間  $[L, R)$  がひとつのブロックに収まる場合以外では、クエリに  $O(1)$  時間で答えられる。

ことが分かりました。したがってここからは、クエリで与えられる区間がひとつのブロックに収まる場合への対応を考えることになります。

Sqrt Tree は、4.1 節の平方分割を再帰的に適用して得られる木構造です。

- 区間  $[0, N)$  を「レベル 0 の区間」とします。
- 非負整数  $d$  に対して、レベル  $d$  の区間を平方分割してできる区間を「レベル  $(d+1)$  の区間」として、レベル  $1, 2, 3, \dots$  の区間を定めます。
- ただし、区間の長さが適当なしきい値 (例えば 2) に到達したところで打ち切ります。

例えば、長さ 16 の列に対して Sqrt Tree は次の図のような木構造です。



## 事前計算

Sqrt Tree の葉を除くすべてのノードに対して、4.1 節で述べた事前計算を行います。つまり、レベル  $d$  の区間において

- 各要素  $A_i$  に対して、 $A_i$  の属するレベル  $d + 1$  の区間を考えて、次の 2 つを計算する。
  - 区間の先頭から  $A_i$  までの総積、
  - $A_i$  から区間の末尾までの総積。
- レベル  $d + 1$  の区間の総積を並べた列に対して、そのすべての部分区間の総積。

を計算します。

## クエリ処理

$d = 0, 1, 2, \dots$  の順に、

- クエリ区間がひとつのレベル  $d + 1$  区間に含まれないならば、4.1 節で説明したように、 $O(1)$  時間でクエリに答えることができる。
- そうでなければ、クエリ区間はひとつのレベル  $d + 1$  区間に含まれる。このノードに進む。

というようにして、木構造を潜っていきます。葉に到達した場合には、愚直に総積を計算します。葉の区間の長さを  $O(1)$  にしておくことで、どのレベルでクエリ処理を終えた場合にも、 $O(1)$  回のモノイド演算でクエリに答えられます。

## 計算量解析

Sqrt Tree の高さを  $H$  としましょう。このとき、

- 各  $d = 0, 1, \dots, H - 1$  について、レベル  $d$  の区間の長さを  $n_1, n_2, n_3, \dots$  とするとき、各区間について  $O(n_i)$  時間の事前計算を行うことになります。  $d$  を固定するとこの総和は  $O(N)$  であり、全体では  $O(NH)$  時間です。空間も同様です。
- クエリ処理では、モノイド演算の回数は  $O(1)$  でした。これに木構造の探索の計算量  $O(H)$  時間が加わり、クエリ処理は  $O(H)$  時間です。

Sqrt Tree の高さ  $H$  はどのようになるのでしょうか？これは、 $N$  から始めて、平方根をとる操作を繰り返して適当なしきい値に到達するまでにかかる回数です。ここから  $H = O(\log \log N)$  であることが分かります。

### ▶ $H = O(\log \log N)$ となる理由

以上により、Sqrt Tree を用いると、【問題 1】は、

- 空間  $O(N \log \log N)$ 、時間  $O(N \log \log N)$  の事前計算のもと
- クエリあたり  $O(\log \log N)$  時間

で解けることが分かります。全体では  $O((N + Q) \log \log N)$  時間ということになります。

## 4.3. クエリ処理の高速化

少しの工夫によって、クエリあたりの計算量を  $O(1)$  とすることも可能です。

計算量を  $O(1)$  にするためには、クエリ区間がどのレベルの区間に含まれるかを知ることができればよいです。

各  $d$  に対して、レベル  $d$  の区間の長さを、最も右側にあるものを除いて適当な 2 のべき乗で揃えるようにします。ブロックの長さがぴったり  $\lfloor \sqrt{N} \rfloor$  にはなりませんが、その定数倍の範囲におさめれば計算量オーダーは変わりません。

このとき、0-based index を用いれば、クエリ区間が  $[L, R)$  であるときに  $L \oplus (R - 1)$  ( $\oplus$  はビット単位 XOR) の値によって、クエリ区間がどのレベルの区間に含まれるかが決まるようになります。したがって、 $L \oplus (R - 1)$  の値ごとに対応するレベルを計算しておく、あるいはビット演算を適切に用いることで、クエリあたりの計算量が  $O(1)$  になります。

この部分の議論については、[Disjoint Sparse Table](#) とも同様です。 [Disjoint Sparse Table](#) や以下の実装例、[文献 2](#)などを参照してください。

## 4.4. 実装例

以下は Library Checker [Static RMQ](#) における実装例です。

- <https://judge.yosupo.jp/submission/335223>

## 5. その他のデータ構造との比較

セグメント木, DST, Sqrt Tree の計算量を比較すると、次の表のようになります。

| データ構造     | 空間                 | 時間                     |
|-----------|--------------------|------------------------|
| セグメント木    | $O(N)$             | $O(N + Q \log N)$      |
| DST       | $O(N \log N)$      | $O(N \log N + Q)$      |
| Sqrt Tree | $O(N \log \log N)$ | $O(N \log \log N + Q)$ |

$N = Q$  程度の状況では Sqrt Tree がセグメント木, DST よりも計算量オーダーが良いことが分かります。特に DST と Sqrt Tree の比較では、計算量オーダーの意味では Sqrt Tree が優位となります。

一方で、DST が区間クエリに対してモノイド演算 1 回、Sqrt Tree が区間クエリに対してモノイド演算 2 回を要することから、どの問題でも Sqrt Tree が DST の完全上位互換として使えるわけではありません。

## 6. 関連問題

1. <https://judge.yosupo.jp/problem/staticrmq>
2. <https://qoj.ac/contest/1356/problem/7182>
3. <https://codeforces.com/contest/1423/problem/C>

## 7. まとめ

本講座では、静的なモノイド列に対する区間積クエリを、Sqrt Tree を用いて処理する方法を解説しました。5 節で見たように  $N = Q$  程度の状況ではセグメント木や DST よりも良い計算量オーダーを実現しており、アイデアも簡単で、面白いと思います。

なお、このように列全体をいくつかのブロックに分割し、区間クエリをブロック境界との位置関係で場合分けして処理する（特にブロックに完全に含まれるものの処理が問題となる）という手法は、線形時間 LCA、線形時間 RMQ にも現れますし、本講座と同様の区間積クエリを  $O((N + Q)\alpha(N))$  時間で処理する方法へと一般化することも可能です（静的な列の区間積クエリ）。

## 8. 参考文献

1. Noga Alon, Baruch Schieber. Optimal preprocessing for answering on-line product queries. arXiv:2406.06321. 2024.  
<https://arxiv.org/abs/2406.06321>.
2. gepardo. Sqrt-tree: answering queries in  $O(1)$  with  $O(N\log\log N)$  preprocessing. Codeforces Blog. <https://codeforces.com/blog/entry/57046>.  
(閲覧日: 2026-03-02).
3. gepardo. Sqrt-tree (part 2): modifications in  $O(\sqrt{N})$ , lazy propagation. Codeforces Blog. <https://codeforces.com/blog/entry/59092>. (閲覧日: 2026-03-02).
4. cp-algorithms. Sqrt Tree. [https://cp-algorithms.com/data\\_structures/sqrt-tree.html](https://cp-algorithms.com/data_structures/sqrt-tree.html). (閲覧日: 2026-03-02).
5. convexineq. 半群の区間積クエリについて.  
<https://qiita.com/convexineq/items/4b6920af3e3d37a96540>. (閲覧日: 2026-03-02).
6. sotanishy. Sqrt Treeで爆速区間クエリ.  
<https://qiita.com/sotanishy/items/89f4dd452bcddd332f24>. (閲覧日: 2026-03-02).
7. maspy. [Library Checker] Static RMQ. <https://maspy.py.com/library-checker-static-rmq>. (閲覧日: 2026-03-02).



8. Wikipedia. Range query (computer science).

[https://en.wikipedia.org/wiki/Range\\_query\\_\(computer\\_science\)](https://en.wikipedia.org/wiki/Range_query_(computer_science)). (閲覧日: 2026-03-02).

なお本講座では Sqrt Tree の用途を、静的な列に対する区間クエリに限定しましたが、文献 2, 3 では、要素の更新などについても解説されています。

---

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)

---



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

## AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

## 各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[アクセスガイド](#)

[AtCoderCareerDesign](#)

[お知らせ](#)

## AtCoder株式会社について

### 資料請求

[会社概要](#)

[AtCoder](#)

[FAQ](#)

[AtCoderJobs](#)

[プライバシーポリシー](#)

[利用規約](#)